

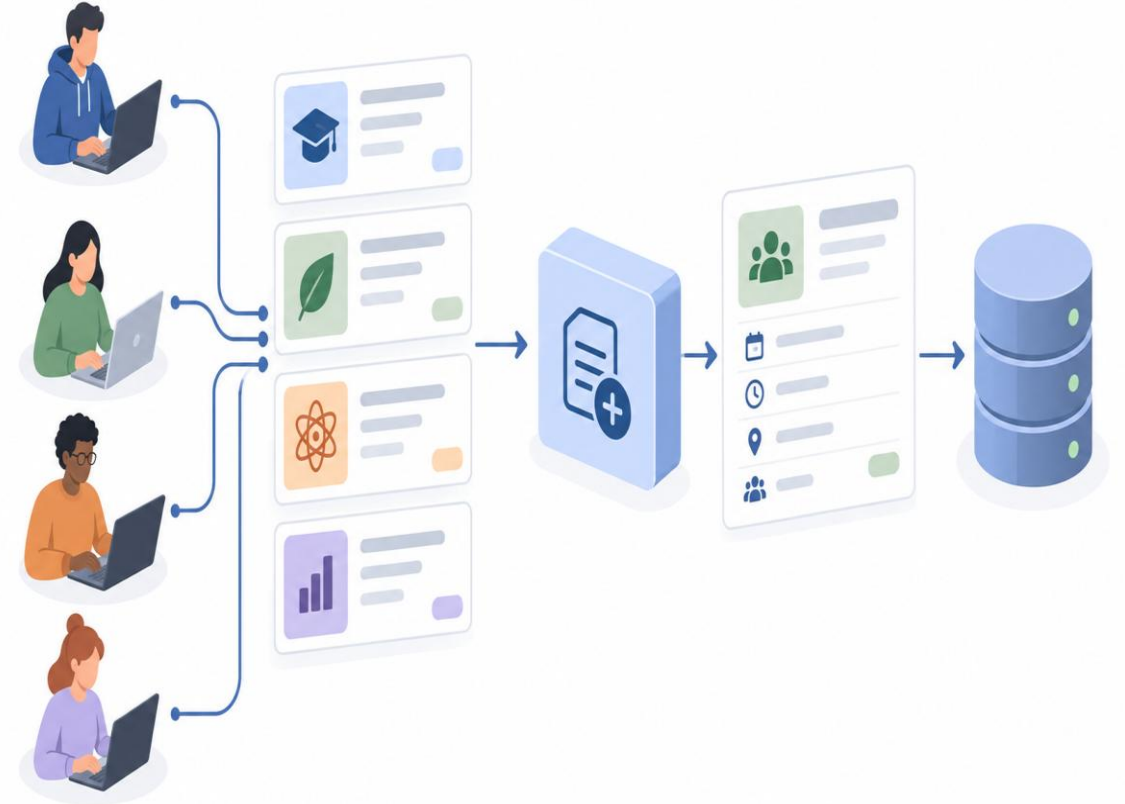
OO Problem Solving

Requirements, classes and collaboration

Discover meaningful classes and give each object a clear responsibility.

Dr. Muhammad Shahid Bhatti

Assistant Professor
Department of Computer Science
COMSATS University Islamabad, Lahore Campus



Bridge from Lecture 7

Lecture 6

A constructor establishes valid state when an object is created.

Lecture 7

Instance state belongs to an object. Static state belongs to the class.

Lecture 8

Before writing classes, decide which concepts and responsibilities the problem needs.

Today: a written requirement becomes an object model.

Lecture 8 scope

- 01** Identify candidate objects and classes from requirements.
- 02** Use nouns and verbs to discover concepts and actions.
- 03** Assign responsibilities and collaborators using CRC cards.
- 04** Sketch and refine an initial conceptual class diagram.

Formal UML notation comes later. Today, the model must be clear and explainable.

Learning outcomes

By the end, you can...	Evidence of understanding
Discover classes	Explain why a candidate is a class or a value.
Assign responsibilities	Name the object that owns a rule and explain why.
Describe collaboration	Trace how objects complete one use case.
Model and refine	Sketch relationships and revise a weak design.
Connect to Java	Create class skeletons and plan scenario checks.

More classes do not automatically produce a better design.

Why design before code?

Code-first result

One large class may collect unrelated fields, duplicate rules and handle every task.

Design-first questions

What concepts exist?
Who owns each rule?
Which objects must collaborate?

Requirement change: a section must reject duplicate requests.

If the rule has a clear owner, you know where the change belongs.

Course Registration System

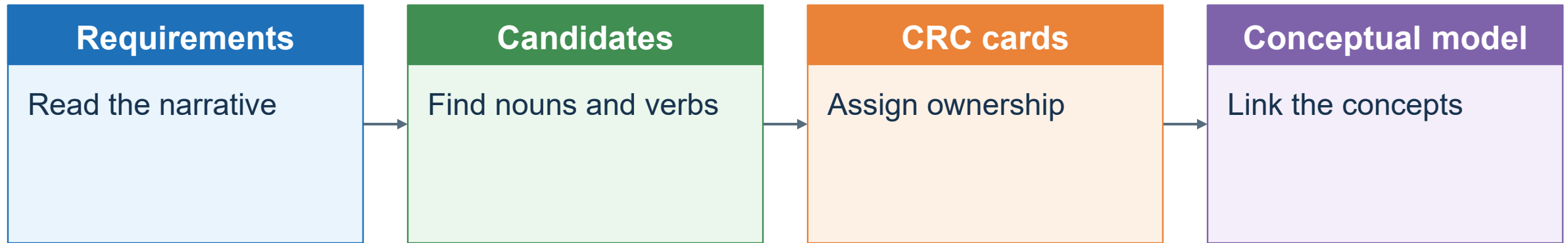
Students browse offered course sections and request enrollment.

- 01** Each section belongs to one course.
- 02** A section has a capacity limit.
- 03** Duplicate enrollment must be rejected.
- 04** Success creates an enrollment record.



First identify the concepts. Implementation details can wait.

OO problem-solving workflow



Scenario check

Can the objects explain success, rejection and a capacity boundary?

Revise the model when the scenario exposes a missing responsibility.

Noun–verb analysis

**“Students request enrollment in course sections.
Each section tracks students and checks capacity.”**

Nouns: candidate concepts

Student, Enrollment, CourseSection
Capacity may be a field.

Verbs: candidate actions

Request, track, check
Decide which object owns each action.

Noun–verb analysis suggests candidates. It does not make the design decisions.

Candidate nouns

Candidate concepts	Candidate values or details
Student	Student ID, name, email
Course	Course code, title, credits
CourseSection	Capacity, section number
Enrollment	Registration time, status
Registration / Semester	List of students, storage structure

Collect candidates first. Their roles remain provisional until you check the context.

Candidate actions

Action in the requirement	Candidate owner or relationship
Browse offered sections	Catalog / application interface
Request enrollment	Student initiates the use case
Check a section's capacity	CourseSection
Prevent duplicate enrollment	CourseSection uses membership records
Create a successful enrollment	Registration coordinates the operation
View current enrollments	Student works with Enrollment records

Not every noun deserves a class

Candidate	Role here	Reason
Student	Class	Individual identity and meaningful state
CourseSection	Class	Owns capacity and membership rules
Enrollment	Class	Relationship with its own state
Capacity / Student ID	Attribute or value	Information owned by another object
List of students	Implementation detail	How we store data, rather than a domain concept
Registration System	System boundary	Too broad to justify one class

Quick check: is “due date” always a class? Explain how the context changes the answer.

Questions for a candidate class

Identity

Abdul Rehman Azam and Ali Ishtiaq represent different Student objects.

State

A CourseSection remembers its capacity and current membership.

Behavior

A CourseSection can accept or reject an enrollment request.

Domain meaning

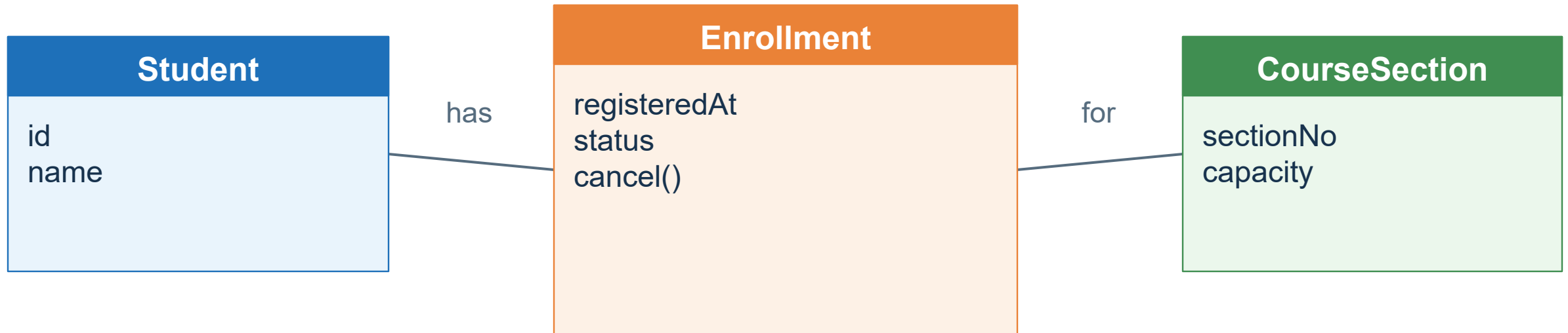
Students and course sections are concepts users recognize.

Attribute or class?

Concept	Simple model	Richer model
Email	String field of Student	Value object if validation or behavior matters
Course	Class with code and title	Still a class, with more rules if needed
Semester	A simple label or value	Class if it owns dates, status and rules

Promote a value when its own behavior or structure makes a separate concept useful.

When a relationship becomes a class



The relationship has its own information and lifecycle.

Enrollment becomes an explicit class because it records more than “Student is linked to Section”.

What an object knows and does

KNOW

Student: identity and name

CourseSection: course, capacity and membership

Enrollment: student, section and status

DO

CourseSection: answer hasSeat() and enforce enrollment rules

Registration: coordinate a request

Student: expose current enrollments

A responsibility describes meaningful knowledge or behavior, beyond a field name.

Encapsulation places rules with data

Rule repeated in client code

```
class CourseSection {  
    public int capacity;  
    public int enrolled;  
}  
  
// Each client repeats this:  
if (section.enrolled <  
    section.capacity) {  
    section.enrolled++;  
}
```

Capacity rule owned by CourseSection

```
class CourseSection {  
    private int capacity;  
    private int enrolled;  
    boolean reserveSeat() {  
        if (enrolled >= capacity)  
            return false;  
        enrolled++;  
        return true;  
    }  
}
```

This slice enforces capacity. Duplicate rejection also needs student membership records.

CRC cards

Class name	
RESPONSIBILITIES What does this class know? What actions or rules does it own? Keep the responsibility list focused.	COLLABORATORS Other class A Other class B

C Class

R Responsibilities

C Collaborators

CRC is a quick way to discuss ownership before writing Java code.

A CRC card in a design discussion

CourseSection	
RESPONSIBILITIES	COLLABORATORS
Know course and capacity.	Course
Know active student membership.	Student
Enforce capacity and duplicate rules.	Enrollment
Accept or reject a request.	

- 01** Name the class.
- 02** Give it core responsibilities.
- 03** Identify needed objects.
- 04** Move a rule to a better owner.

If the card needs unrelated responsibilities, reconsider the class boundary.

CRC example: Student

Student	
RESPONSIBILITIES	COLLABORATORS
Know registration ID and name.	Enrollment
Know current enrollment records.	Registration
Initiate a registration request.	

Example object

Abdul Rehman Azam

SP26-BAI-003

Student starts the request. The section still owns its own capacity rule.

CRC example: CourseSection

CourseSection	
RESPONSIBILITIES	COLLABORATORS
Know course, capacity and membership. Answer whether a seat is available.	Course Student
Reject duplicate active membership.	Enrollment
Accept enrollment only if rules pass.	

Section A

capacity = 2
active students = 2

New request
Result: reject

A section protects its rules at the point where membership changes.

CRC example: Enrollment

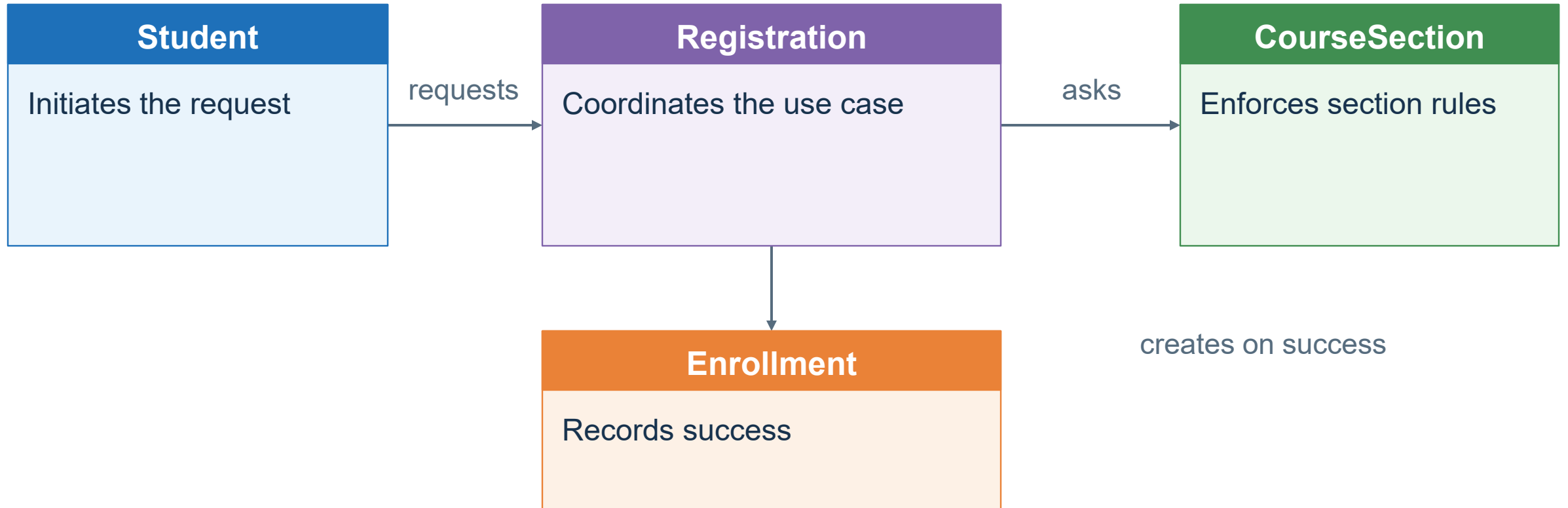
Enrollment	
RESPONSIBILITIES	COLLABORATORS
Connect one student and one section.	Student
Record registration time and status.	CourseSection
Represent a successful registration.	

One record

student: Ali Ishtiaq
section: CSC241-A
status: active

Enrollment stores the meaning of the relationship in one coherent object.

Object collaboration

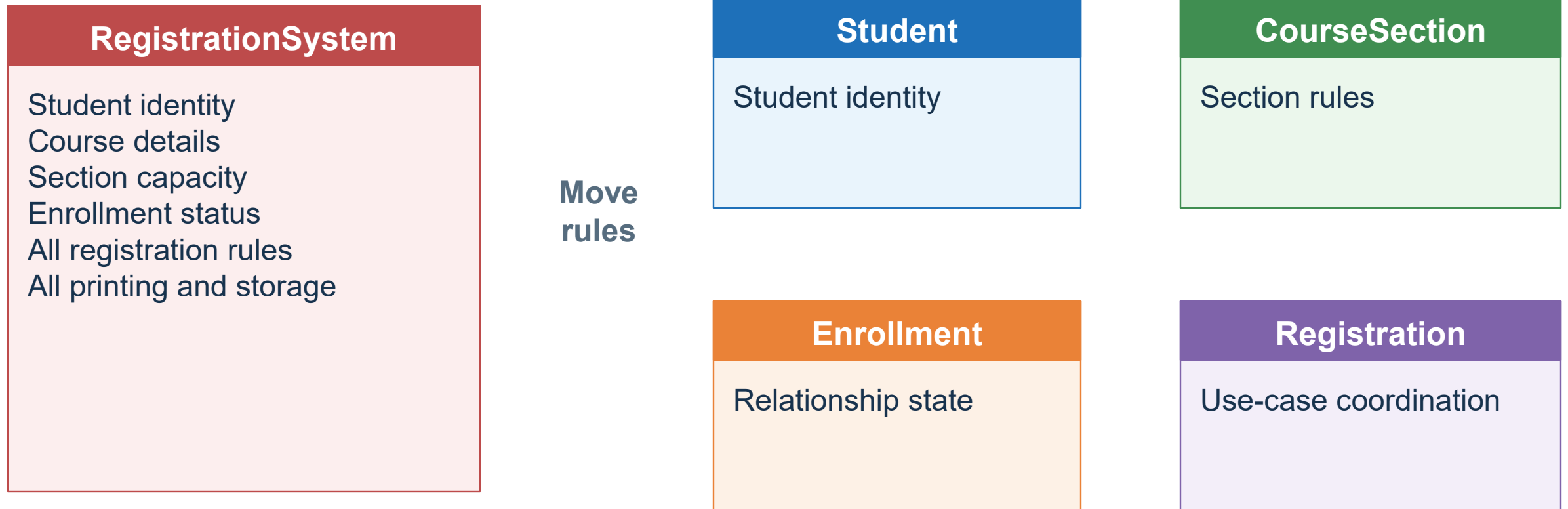


Who owns a responsibility?

Question or action	Owner	Why
Is this section full?	CourseSection	Owns capacity and active membership
What is the student's ID?	Student	Owns student identity
Has this student enrolled already?	CourseSection	Owns membership for this section
Coordinate a registration request	Registration	Arranges work across objects
Record registration time and status	Enrollment	Owns the relationship's state

Ownership question: which object already has the information needed for this decision?

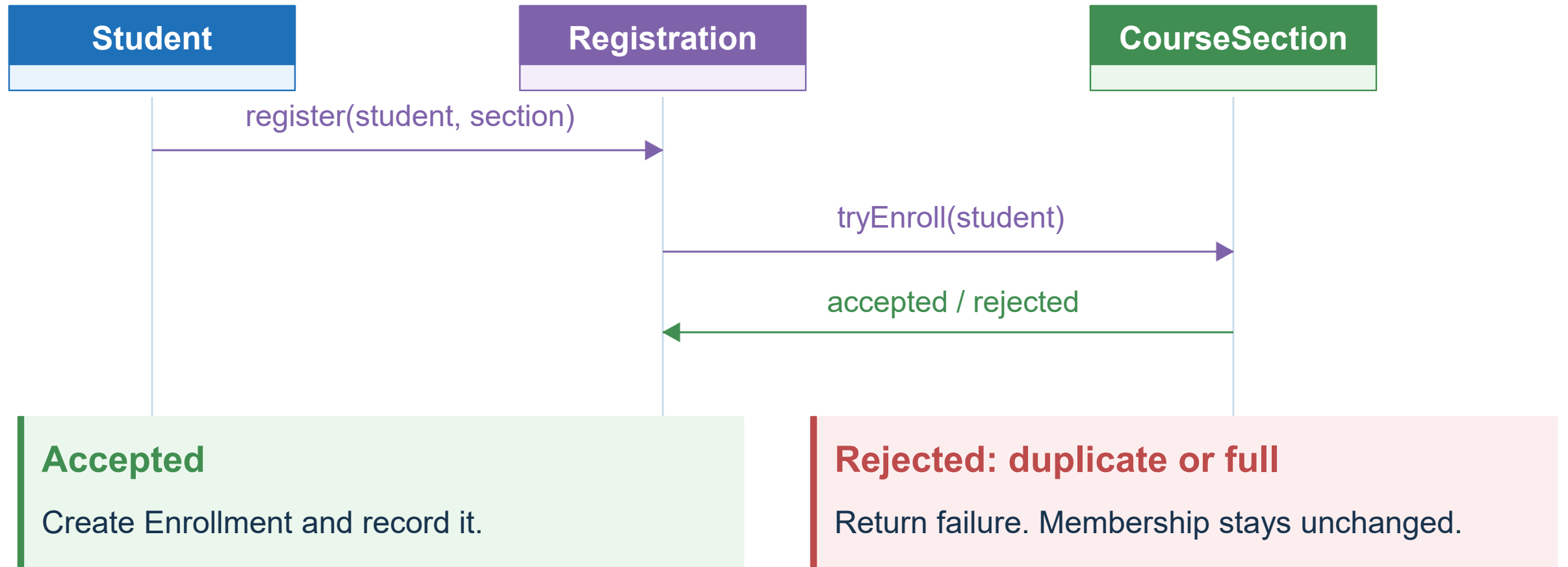
The overloaded class



A coordinator can be useful. It should delegate rules to the objects that own them.

A registration interaction

Ali Ishtiaq requests CSC241 section A.



Successful enrollment: before and after

Teaching example: Ali Ishtiaq requests CSC241 section A.

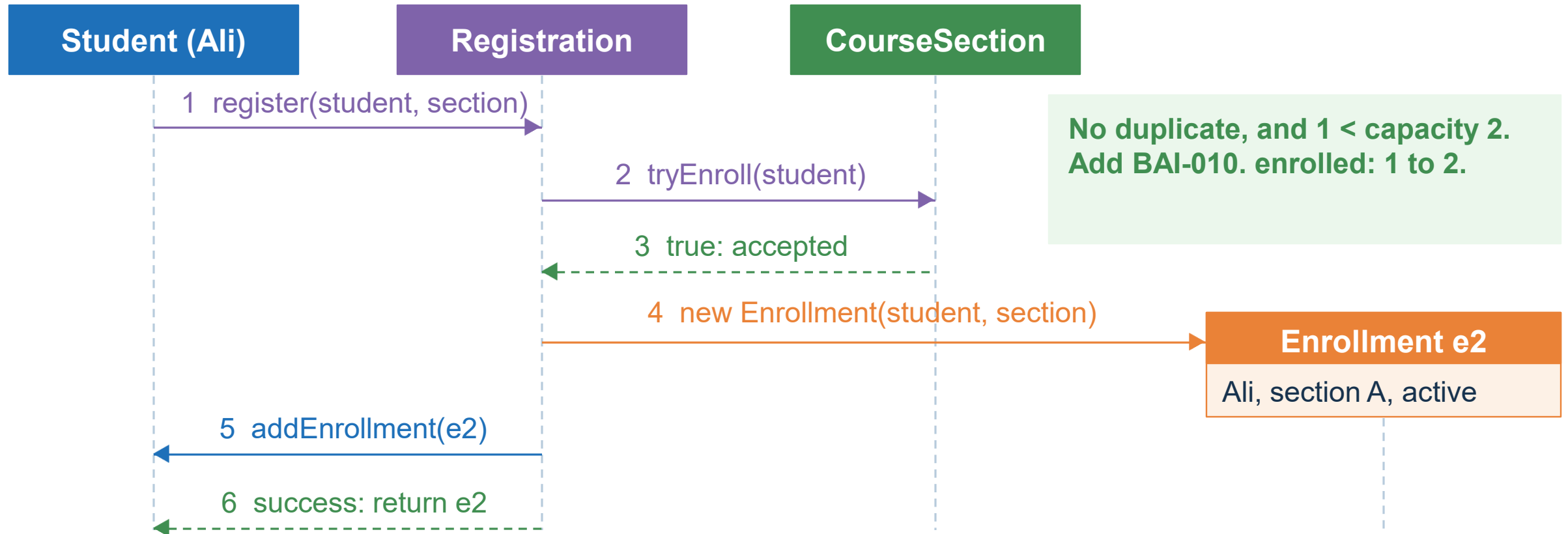
Before the request, Abdul Rehman Azam occupies the section's first seat.

State being observed	Before request	After success
Section capacity	2	2 (unchanged)
Section enrolled count	1	2
Section active student IDs	{BAI-003}	{BAI-003, BAI-010}
Ali's current enrollments	[]	[e2]
Enrollment object e2	Does not exist	Exists: Ali, section A, active
section.hasSeat()	true ($1 < 2$)	false ($2 < 2$ is false)

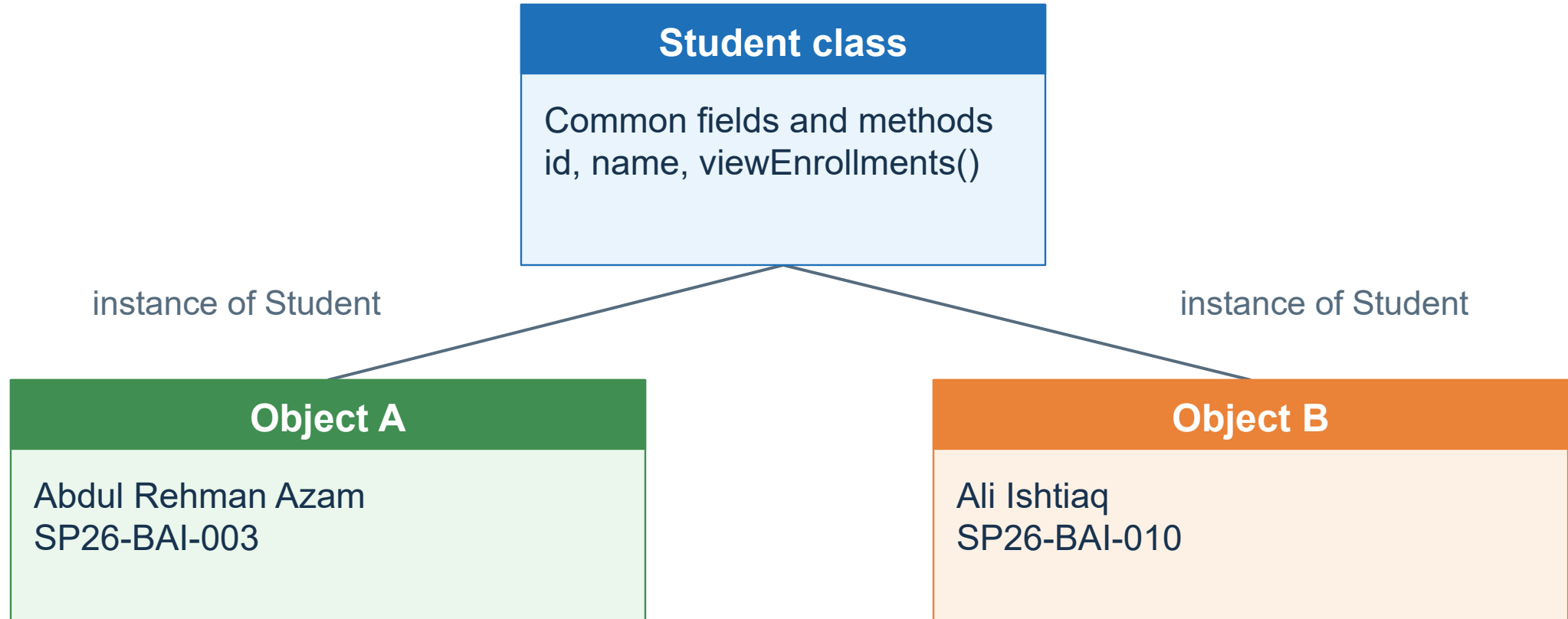
Student identity and course details stay the same. Membership and enrollment records change.

Successful enrollment: sequence diagram

Messages run top to bottom. Each object updates the state it owns.



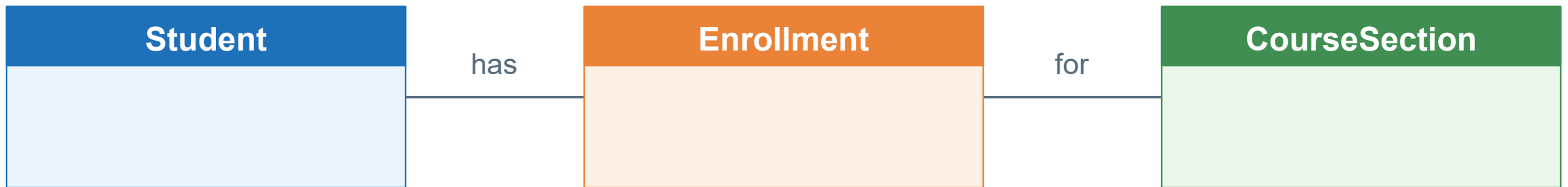
Class and object thinking



Both objects follow one class definition. Each has its own identity and state.

A conceptual class diagram

A conceptual diagram shows domain concepts and their meaningful relationships.



Keep Java collection types, database tables and interface widgets out of the first sketch.

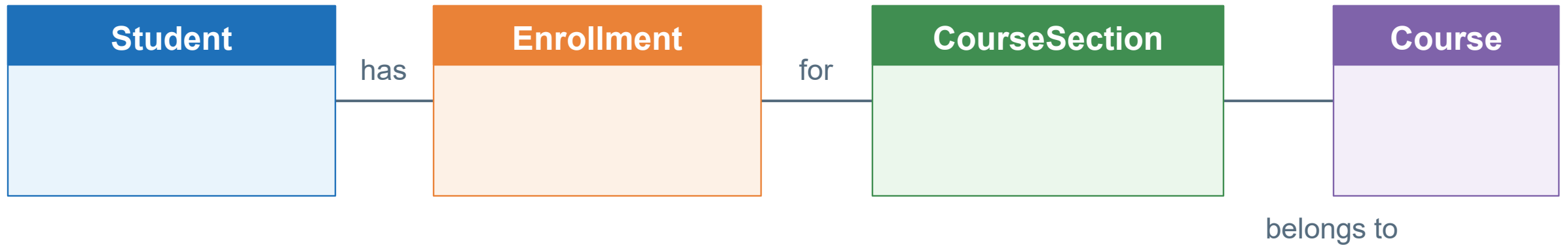
The diagram is a shared model of the problem, before detailed implementation.

Lightweight diagram notation



Element	Meaning
Box	A concept in the domain
Line	A meaningful association
Label	Domain wording such as “belongs to” or “has”

Course-registration model



One Enrollment connects one Student with one CourseSection.

Registration coordinates the use case. It is a service in this design, outside the domain association chain.

Students and sections can each be linked to multiple Enrollment records.

Verbs and relationships

Domain wording	Relationship or responsibility?
CourseSection belongs to Course	Structural relationship
Student has Enrollment records	Structural relationship
Enrollment is for CourseSection	Structural relationship
CourseSection checks capacity	Behavior owned by CourseSection
Registration coordinates a request	Use-case collaboration

A verb may describe a lasting relationship or an action that happens during a use case.

Refining the model

Remove as classes

Capacity: a field

Student ID: a value

ListOfStudents: storage
detail

Keep as concepts

Student

Course

CourseSection

Enrollment

Revisit if needed

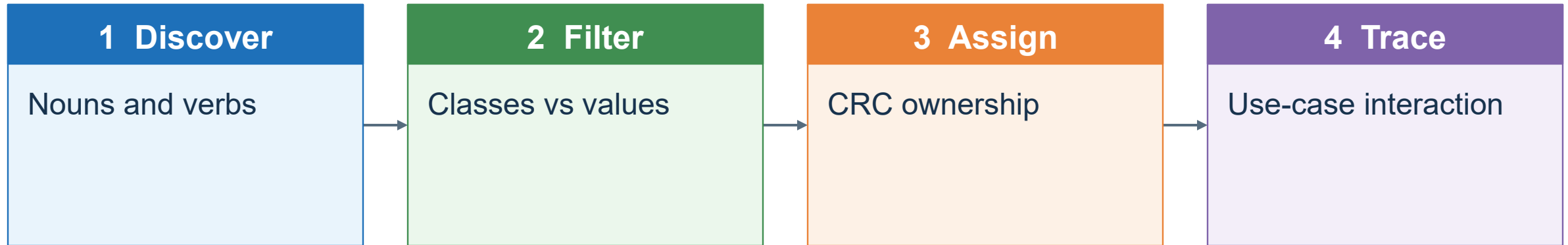
Registration coordinator

Semester

Prerequisite rules

A revised model should follow the requirements, rather than add speculative concepts.

Design as an iterative process



Feedback: a scenario exposes a missing rule or collaborator.

Revise the model, then code a small slice and check the scenarios.

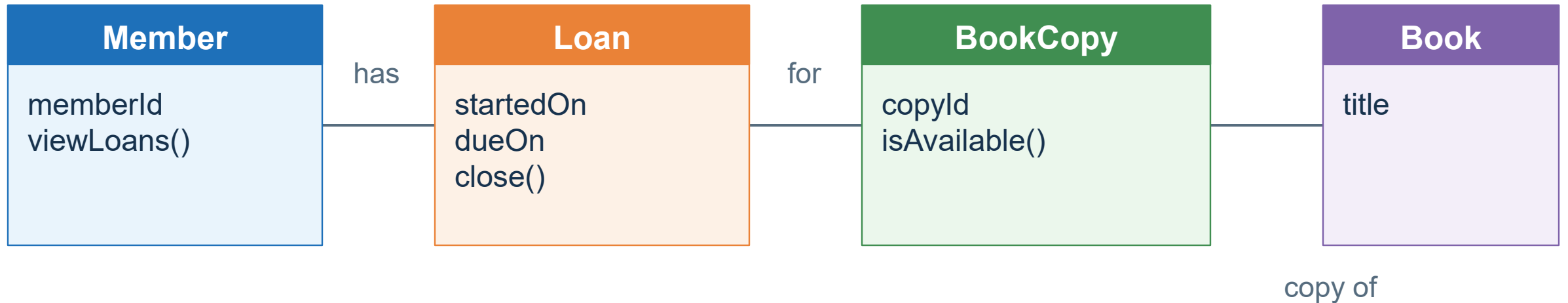
Practice: Library borrowing

A library member borrows an available book copy.
A copy can have at most one active loan.
The loan records its start date and due date.
Members can view loans and return copies.

- 01 Find candidate classes and values.
- 02 Assign borrowing rules to an owner.
- 03 Draft three CRC cards and sketch relationships.

5 minutes in pairs: defend one noun you rejected as a class.

A library model



Book: title-level information

BookCopy: one physical copy with its own availability

Loan records the borrowing relationship. Due date remains a value here.

Java class skeletons

Structural skeletons only. Constructors and registration behavior come next.

```
class Student {  
    private String id;  
    private String name;  
}
```

```
class Course {  
    private String code;  
    private String title;  
}
```

```
class CourseSection {  
    private Course course;  
    private int capacity;  
    private int enrolled;  
    boolean hasSeat() {  
        return enrolled  
            < capacity;  
    }  
}
```

```
class Enrollment {  
    private Student student;  
    private CourseSection  
        section;  
}  
  
class Registration {  
    // behavior to add  
}
```

A compiling skeleton does not yet satisfy capacity and duplicate-enrollment requirements.

Live coding and scenario checks

Create the class files and a Main.java driver in one simple folder.

```
javac *.java  
java Main
```

Keep these class files and Main.java in one folder for this example.

Scenario	Expected behavior
New student, available seat	One success record; active membership increases by 1
Same student requests the same section again	Reject; state stays unchanged
Section is already full	Reject; no successful enrollment is created

Team exercise: Campus parking

A student registers a vehicle and requests a parking permit.

A permit is valid for one vehicle and one semester.

Parking areas have capacity limits.

Students can view active permits.

Your group produces	Measure of completion
Candidate list	At least one rejected value or implementation detail
3–4 CRC cards	Every class has a justified responsibility
Conceptual diagram	Relationship labels explain the requirement
Scenario check	Explain one capacity rejection

Common modeling mistakes

Mistake	Better design question
Every noun becomes a class	Does it need its own identity, behavior or structure?
One class owns everything	Which object naturally owns this information or rule?
Fields replace responsibilities	What meaningful action or decision does the class own?
A diagram must be perfect first	Can a scenario reveal what should change?
Only the success path matters	What happens for duplicates and at capacity?
All lines mean the same thing	Is this a domain association or a use-case action?

Design checklist and exit ticket

Design checklist

- 01 Concepts and values have clear roles.
- 02 Responsibilities have clear owners.
- 03 Relationships use domain language.
- 04 The main scenario has a complete path.
- 05 Rejection leaves a valid state.

Exit ticket

- 01 Name a noun you rejected and explain why.
- 02 Give one rule owned by CourseSection.
- 03 Why does Enrollment deserve a class?
- 04 Name one rejection or boundary check.

Next: Object composition, references between objects and has-a relationships.